Project 6 Elevator RPi Setup Scripts

Introduction:

The Elevator Project uses a Raspberry Pi (RPi) computer as a supervisory controller, and it must perform a variety of different tasks to complete its duties within the elevator system. The RPi must communicate with the car, floor, and elevator controllers over a shared CAN buss using the PEAK CAN driver and adapter; process requests, log system information and request records in a MySQL database; host the database and a website on the local network to enable remote control of the system using a web-based GUI. All of these functions require supporting architecture and setup in order to function optimally, and the setup process is fairly involved. Because of the length of the setup process, it is possible that some required settings are missed and go unconfigured upon system setup. Since we are sharing the physical hardware platform with another group, we should try to work within a different environment on the RPi as much as we can, and we cannot be sure that all of the settings remain in the required state. For these reasons, we decided to script the setup process for each of the foundational steps (the RPi setup, server setup, and database setup) using Bash to ensure that our working environment remains up to date, properly configured, and that it is easy to refresh if something goes wrong.

Equipment Description:

The RPi that we will be using is an RPi 3B, which will be connected to the provided wireless elevator network, and by CAN to the floor, car, and elevator controllers. Most of the setup steps that are performed in these setup scripts install dependency packages, establish filesystems, create or modify configuration files, or prompts the user to make adjustments where it is easier than automation. The scripts will set up the RPi environment, set up MySQL database, and configure the apache2 server for hosting the website and database.

The set of setup scripts included in this project are meant to simplify and streamline critical environment setup work that takes a long time, is tedious, and may easily include errors into the project. By scripting these processes, we remove an element of certainty and make an easily reproducible environment. These scripts include:

scrip_auth.sh:

script_auth.sh is the first script that should be run. This script:

- Establishes a home directory for further scripting
- Creates files to save logs into
- Gives system-wide permissions to the files inside that directory
- Removes password restrictions for setup scripts to ensure that they can do their thing without issues.

rpi_setup.sh and log_rpi_setup.sh:
rpi_setup.sh:

- Installs all required dependencies
- Configures the PEAK CAN adapter drivers
- Sets bitrates for CAN
- Sets up system settings for ssh, vnc, I2C, and SPI

log_rpi_setup.sh:

- Runs rpi_setup.sh
- Logs the stdout and stderr data streams to the log file created in script_auth.sh

server_setup.sh and log_server_setup.sh:
server_setup.sh:

- Configures apache2 and initializes a server location
- Configures myphpadmin
- Sets up ufw firewall and shh access

log_server_setup.sh:

- Runs server_setup.sh
- Logs the stdout and stderr data streams to the log file created in script_auth.sh

database_setup.sh and log_database_setup.sh:
database_setup.sh:

- Configures mysql for use with myphpadmin
- Sets up access permissions and passwords
- Initializes a new database and table for datarecording

log_database_setup.sh:

- Runs database_setup.sh
- Logs the stdout and stderr data streams to the log file created in script_auth.sh

GUI Design Decisions:

The initial intention was to include all of these functions in one master setup script, but it became evident that this was not optimal. It is more advantageous to have the setup script separated by general target or function so that the whole system does not need to be reconfigured to redo one aspect of the setup. Additionally, it makes reading the log files much easier, and prevents setup for one aspect from preventing the setup of another if something goes wrong and the script fails. Further, we decided to

provide instructions and automatic entry into large files that needed small changes, so that these files do not become compromised by mistargeting a setting – it's just safer.